

# EXPERIMENT – 11

**NAME: K ANUSRI**

**ROLL NO: 2520030432**

## QUESTION 11

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

## AIM

To implement a command history buffer and pipeline structure, handle capacity limits, retrieve and validate commands, maintain pipeline execution order, connect processes, and test the configuration.

## THEORY

A command history buffer stores previously entered commands for retrieval and display. A fixed-size buffer handles capacity limits by removing the oldest entry when full. A pipeline connects processes so data can pass between them using pipe(). Process creation and synchronization are demonstrated using fork() and wait().

## ALGORITHM

1. Initialize the history buffer and pipeline structure.
2. Store and display commands.
3. Retrieve a history entry.
4. Test the history capacity limit.
5. Validate the history buffer.
6. Store pipeline commands in execution order.
7. Validate the pipeline layout.
8. Create processes using fork().
9. Connect processes using pipe().
10. Transfer data and synchronize processes using wait().
11. Display the final test result.

## C PROGRAM

```
k_anusri@DESKTOP-TM20BAQ:~$  
k_anusri@DESKTOP-TM20BAQ:~$ #include <stdio.h>  
#include <string.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
#define MAX 5  
  
char history[MAX][50];  
int hcount = 0;  
char pipeline[MAX][30];  
int pcount = 0;  
  
void add_history(char *cmd)  
{  
    int i;  
    if (hcount == MAX)  
    {  
        for (i = 1; i < MAX; i++)  
            strcpy(history[i-1], history[i]);  
        hcount--;  
        printf("History capacity reached. Oldest entry removed.\n");  
    }  
    strcpy(history[hcount++], cmd);  
}  
  
void show_history()  
{  
    int i;  
    printf("\nCommand History:\n");  
    for (i = 0; i < hcount; i++)  
        printf("%d. %s\n", i + 1, history[i]);  
}  
  
void pipeline_test()
```

```
{
    int fd[2];
    pid_t p1, p2;
    char msg[] = "Pipeline Data";
    char buf[50];

    pipe(fd);
    p1 = fork();

    if (p1 == 0)
    {
        close(fd[0]);
        write(fd[1], msg, strlen(msg) + 1);
        close(fd[1]);
        printf("Process 1: Command executed.\n");
        return;
    }

    p2 = fork();

    if (p2 == 0)
    {
        close(fd[1]);
        read(fd[0], buf, sizeof(buf));
        close(fd[0]);
        printf("Process 2: Received -> %s\n", buf);
        return;
    }

    close(fd[0]);
    close(fd[1]);
    wait(NULL);
    wait(NULL);
    printf("Pipeline execution completed successfully.\n");
}
```

```
int main()
{
    int i;
    char *cmds[] = {"pwd", "ls", "cd /home", "echo Hello", "date"};

    printf("=====\n");
    printf(" COMMAND HISTORY AND PIPELINE TEST\n");
    printf("=====\n");

    printf("\n1. COMMAND HISTORY\n");
    printf("-----\n");
    for (i = 0; i < 5; i++)
        add_history(cmds[i]);
    show_history();

    printf("\n2. RETRIEVE HISTORY ENTRY\n");
    printf("-----\n");
    printf("Entry 3: %s\n", history[2]);

    printf("\n3. CAPACITY LIMIT TEST\n");
    printf("-----\n");
    add_history("whoami");
    show_history();

    printf("\n4. HISTORY VALIDATION\n");
    printf("-----\n");
    if (hcount <= MAX)
        printf("History is valid.\n");

    printf("\n5. PIPELINE STRUCTURE\n");
    printf("-----\n");
    strcpy(pipeline[0], "echo");
    strcpy(pipeline[1], "filter");
    strcpy(pipeline[2], "display");
    pcount = 3;
```

```
printf("\n3. CAPACITY LIMIT TEST\n");
printf("-----\n");
add_history("whoami");
show_history();

printf("\n4. HISTORY VALIDATION\n");
printf("-----\n");
if (hcount <= MAX)
    printf("History is valid.\n");

printf("\n5. PIPELINE STRUCTURE\n");
printf("-----\n");
strcpy(pipeline[0], "echo");
strcpy(pipeline[1], "filter");
strcpy(pipeline[2], "display");
pcount = 3;

for (i = 0; i < pcount; i++)
    printf("Process %d: %s\n", i + 1, pipeline[i]);

printf("\n6. PIPELINE VALIDATION\n");
printf("-----\n");
if (pcount > 0 && pcount <= MAX)
    printf("Pipeline layout is valid.\n");

printf("\n7. PIPELINE PROCESS TEST\n");
printf("-----\n");
pipeline_test();

printf("\n=====");
printf("ALL TESTS COMPLETED SUCCESSFULLY\n");
printf("=====");
return 0;
}
```

## COMPILATION AND EXECUTION

```
gcc q11.c -o q11
./q11
```

## EXECUTION OUTPUT – SCREENSHOT 1

## COMMAND HISTORY AND PIPELINE TEST

### 1. COMMAND HISTORY

Command History:

1. pwd
2. ls
3. cd /home
4. echo Hello
5. date

### 2. RETRIEVE HISTORY ENTRY

Entry 3: cd /home

### 3. CAPACITY LIMIT TEST

History capacity reached. Oldest entry removed.

Command History:

1. ls
2. cd /home
3. echo Hello
4. date
5. whoami

### 4. HISTORY VALIDATION

History is valid.

### 5. PIPELINE STRUCTURE

Process 1: echo  
Process 2: filter  
Process 3: display

## EXECUTION OUTPUT – SCREENSHOT 2

### 6. PIPELINE VALIDATION

-----  
Pipeline layout is valid.

### 7. PIPELINE PROCESS TEST

-----  
Process 1: Command executed.

=====  
ALL TESTS COMPLETED SUCCESSFULLY

=====  
Process 2: Received -> Pipeline Data

=====  
ALL TESTS COMPLETED SUCCESSFULLY

=====  
Pipeline execution completed successfully.

=====  
ALL TESTS COMPLETED SUCCESSFULLY

## RESULT

The command history buffer and pipeline structure were implemented successfully. The program stored and displayed commands, handled the history capacity limit, retrieved entries, validated the history, created and validated a pipeline layout, and demonstrated inter-process communication and synchronization. All tests were completed successfully.